



Universiteit Antwerpen
| Faculteit Toegepaste
Ingenieurswetenschappen

4-Computerarchitectuur en besturingssystemen 2

Overzicht

Studiegids

■ Doel:

- zelfstandig technische documentatie van een microcontroller/microprocessor analyseren en verwerken
- de interne werking van de verschillende onderdelen van de processor simuleren en toelichten (adhv VHDL)
- de theorie van besturingssystemen toepassen in een project op een microcontroller-kit
- de ontwerptools een 'microcontrollersysteem' configureren en debuggen

Opbouw

- **Vier modules:**

- Computerarchitectuur:

- werking van de core mbv microcontroller-kit (assembler): module “core”
 - werking van de periferie mbv microcontroller-kit: module “periferie”
 - interne werking van processor adhv VHDL model (SimpleCPU): module “CPU hardware”

- Besturingssystemen:

- tasks, mutexen, semaphoren, rtos,...: module “OS”

- **Drie groepen:**

- doorschuifstelsel
 - iedere groep volgt alle modules (zie Blackboard voor planning)

Planning

- **15 practica van 4u (vanaf lesweek 3)**
 - 9 sessies computerarchitectuur:
 - 3 sessies core (assembler)
 - 3 sessies CPU hardware (VHDL)
 - 3 sessies periferie (C)
 - 5 sessies Operating Systemen
 - 1 sessie evaluatie
 - per onderdeel: test van 1 uur
- **Minstens 13 zittingen aanwezigheid vereist!**
 - wettig afwezig? => inhalen mogelijk (op initiatief van de student)

Beoordeling

- **Portfolio voor core, periferie en OS**
- **Test tijdens laatste sessie voor alle vier de modules**
- **Eindscore:**
 - core: 20%
 - portfolio (20%) + test (80%)
 - CPU hardware: 20%
 - test (100%)
 - periferie: 20%
 - portfolio (20%) + test (80%)
 - OS: 40%
 - portfolio (20%) + test (80%)

Beoordeling

- **Minimum 13 sessies aanwezigheid, anders max 7/20!**
- **Minimum 30% per onderdeel, anders max 8/20!**
- **Modules waarvoor geslaagd: punten blijven behouden in tweede zit**
- **Geen deelloverdrachten naar het volgend academiejaar**
- **Evaluatiecriteria portfolio:**
 - gestructureerd
 - bevat de verslagen van de opdrachten
- **Test tijdens laatste sessie:**
 - oefeningen gelijkaardig aan die van tijdens de practica

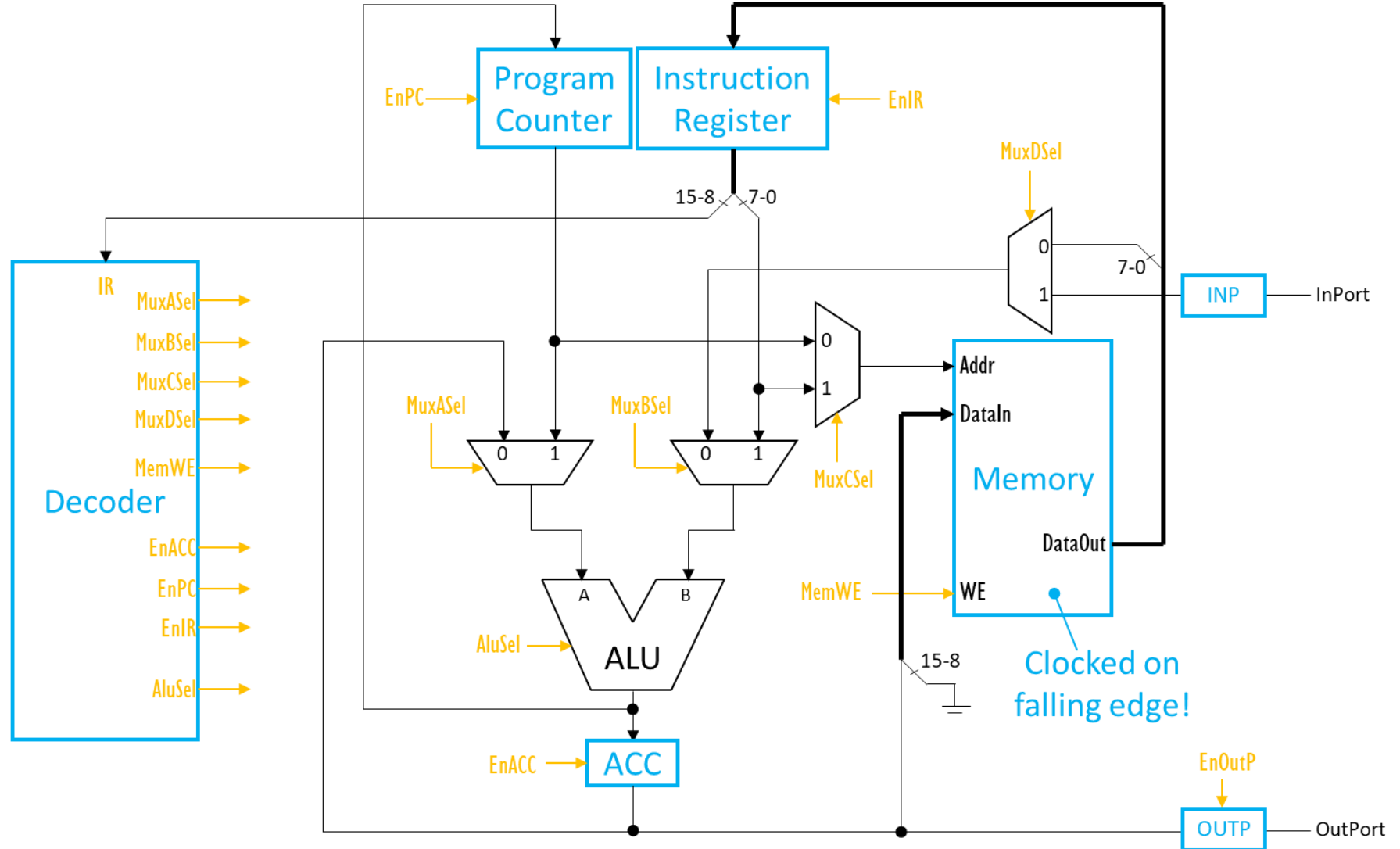
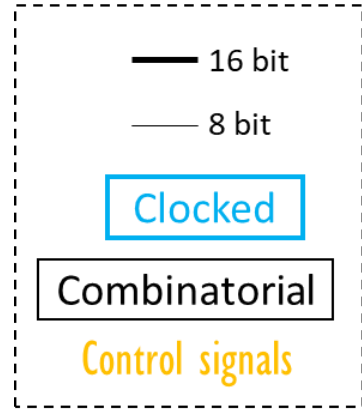
Module CPU hardware

- **Zelf ontworpen processor “SimpleCPU”: VHDL**
 - basiskennis VHDL vereist
 - enkel simulatie, geen configuratie in FPGA: module “core”
 - Xilinx Vivado: versie onbelangrijk
- **Sterk vereenvoudigde CPU:**
 - twintigtal instructies van 16-bits
 - geen subroutines, stack, heap, interrupts,...: andere modules
 - geen pipelining, branch prediction,...: geavanceerde (gepatenteerde) algoritmes
 - gedeeld instructie/data geheugen, geen cache
 - beperkte ALU
 - fetch, decode, execute, increment

Simple CPU instructies

Instructie	Opcode	Operand	Functie
LOAD ACC kk	0000 XXXX	KKKK KKKK	Bewaar de constante K in de accumulator
AND ACC kk	0001 XXXX	KKKK KKKK	Voer een bitgewijze EN uit met K op de accumulator
ADD ACC kk	0010 0XXX	KKKK KKKK	Tel de waarde K op bij de accumulator
ADD IND mm	0010 1XXX	MMMM MMMM	Tel de waarde op geheugenadres M op bij de accumulator
SUB ACC kk	0011 XXXX	KKKK KKKK	Trek de waarde K af van de accumulator
READ MEM mm	0100 XXXX	MMMM MMMM	Stop de waarde van adres M in de accumulator
WRITE MEM mm	0101 XXXX	MMMM MMMM	Schrijf de waarde van de accumulator naar adres M
INPUT ACC	0110 XXXX	XXXX XXXX	Stop de waarde van de input port in de accumulator
OUTPUT ACC	0111 XXXX	XXXX XXXX	Schrijf de waarde van de accumulator naar de output port
JUMP U aa	1000 XXXX	AAAA AAAA	Spring onvoorwaardelijk naar instructie A
JUMP Z aa	1001 00XX	AAAA AAAA	Spring naar instructie A als de Z-vlag '1' is
JUMP C aa	1001 10XX	AAAA AAAA	Spring naar instructie A als de C-vlag '1' is
JUMP NZ aa	1001 01XX	AAAA AAAA	Spring naar instructie A als de Z-vlag '0' is
JUMP NC aa	1001 11XX	AAAA AAAA	Spring naar instructie A als de C-vlag '0' is
JUMP IND aa	1011 XXXX	MMMM MMMM	Spring naar de instructie waarvan het adres op M staat

Simple CPU opbouw



Simple CPU componenten

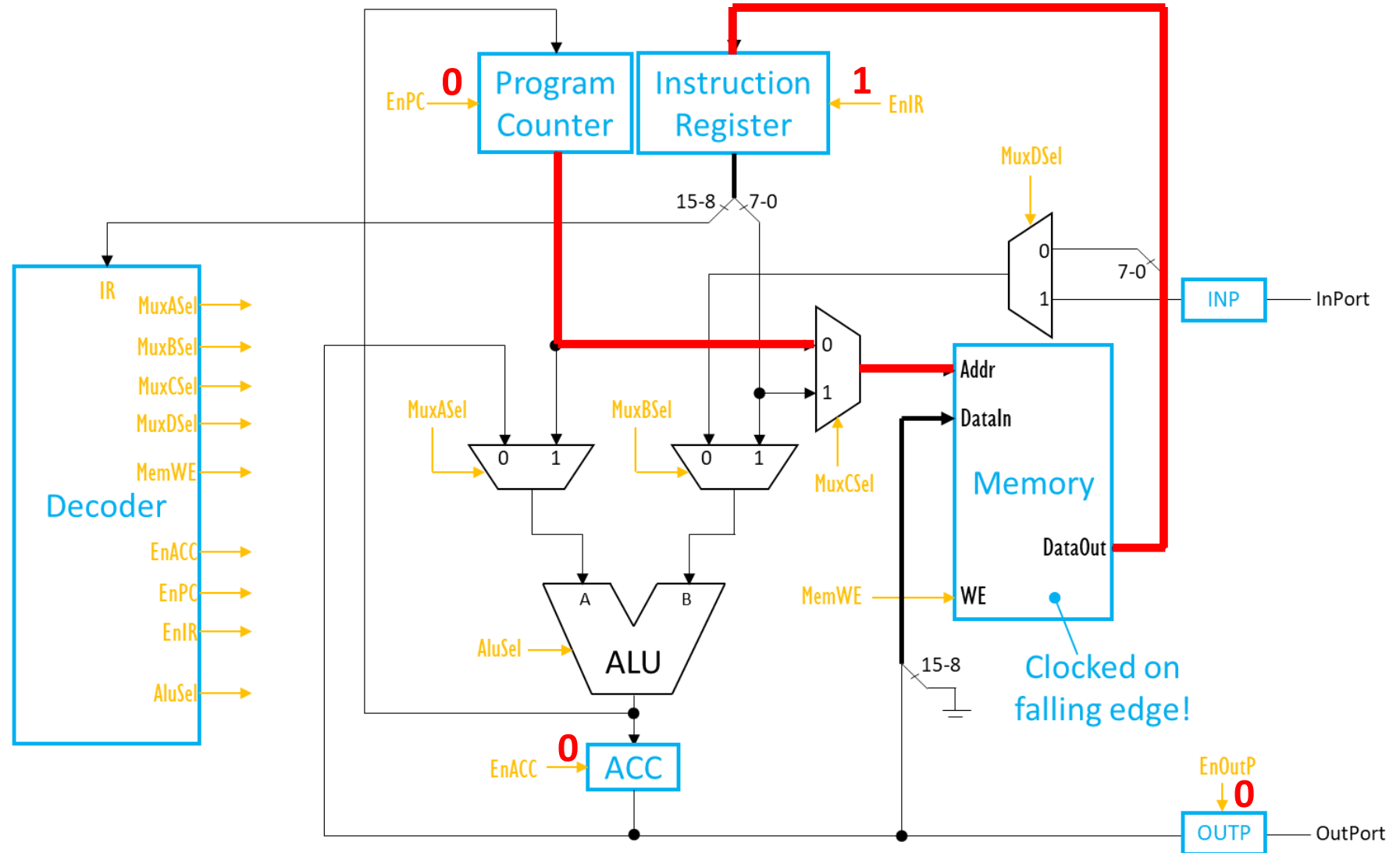
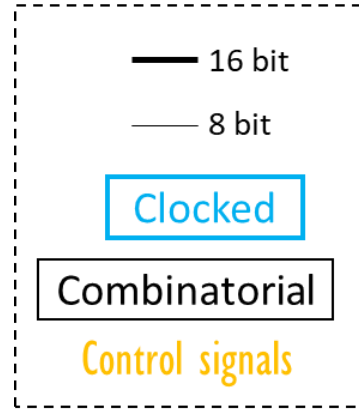
- **Programma- en datageheugen:**
 - address, data in, data out, write enable
- **Andere registers = flip flops: zie VHDL**
 - accumulator: resultaat laatste bewerking
 - program counter: adres huidige instructie
 - instruction register: huidige instructie
 - output register: naar buitenwereld
 - timer registers (niet op blokschema: zie opgaven)
- **Decoder blok:**
 - interpreteert huidige instructie
 - zet alle selectiesignalen multiplexers juist

Simple CPU componenten

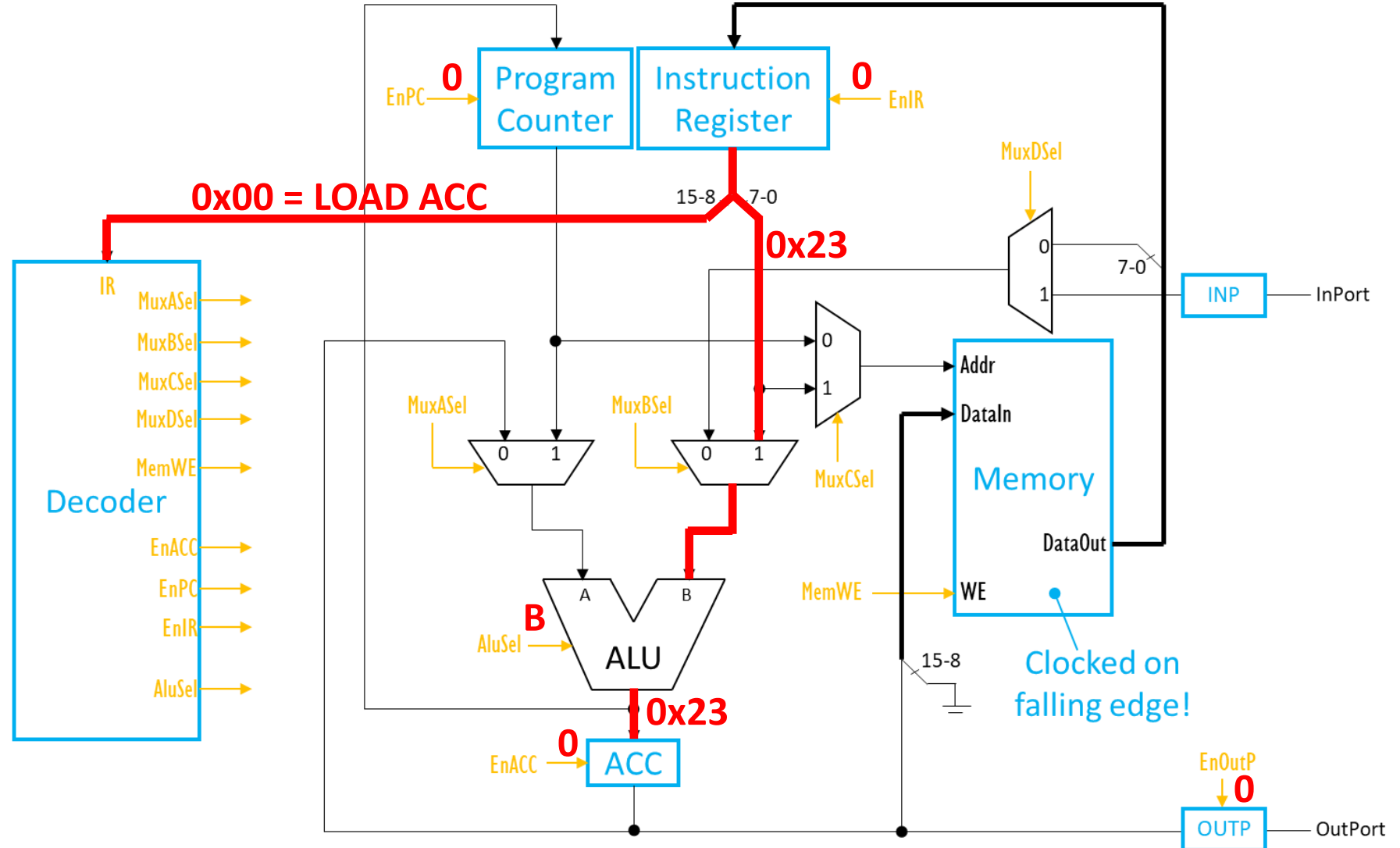
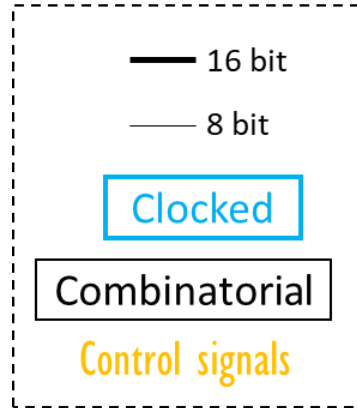
- **ALU: arithmetic logic unit**

- doet allerhande berekeningen
- beschikt over “vlaggen”:
 - carry flag = de laatste carry bit van een bewerking (bv. optelling)
 - let op: $\neq$ altijd overflow (bv. two's complement $A-B$: zie basis digitale elektronica)
 - zero flag = 1 wanneer het resultaat van een bewerking gelijk is aan 0
 - gebruikt voor voorwaardelijke jump instructies
- heeft twee ingangen: A en B
- decoder beslist welke bewerking er gebeurt:
 - A, B, $A+B$, $A-B$, $A \& B$ of $A+1$
 - eventueel andere bewerkingen bij uitbreiding (zie opgaven)

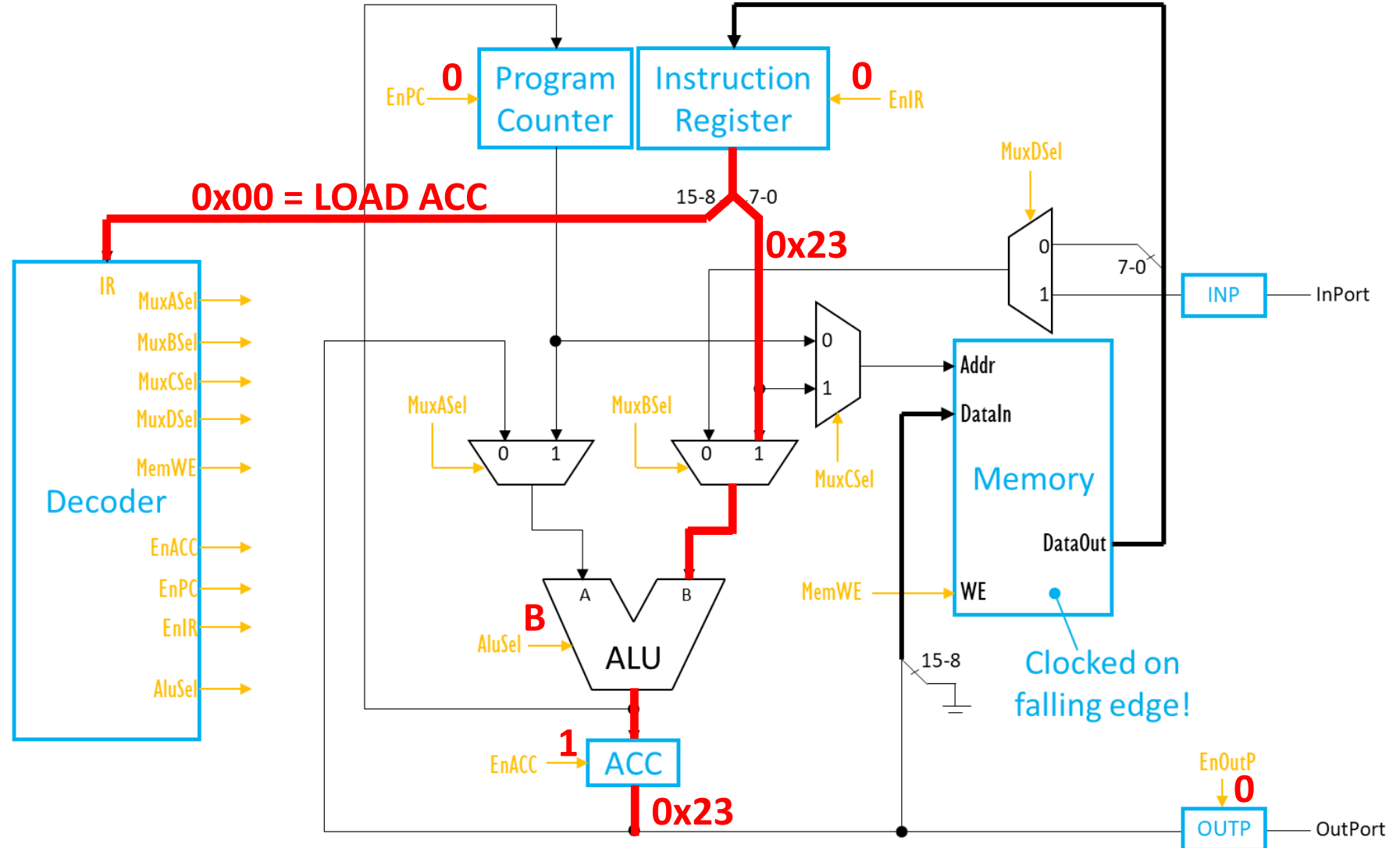
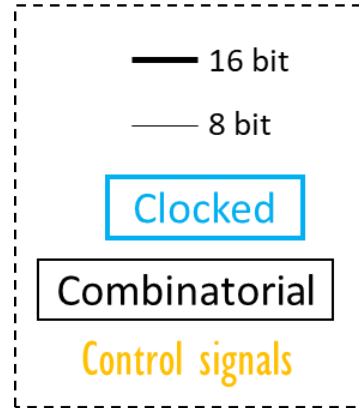
Simple CPU: fetch



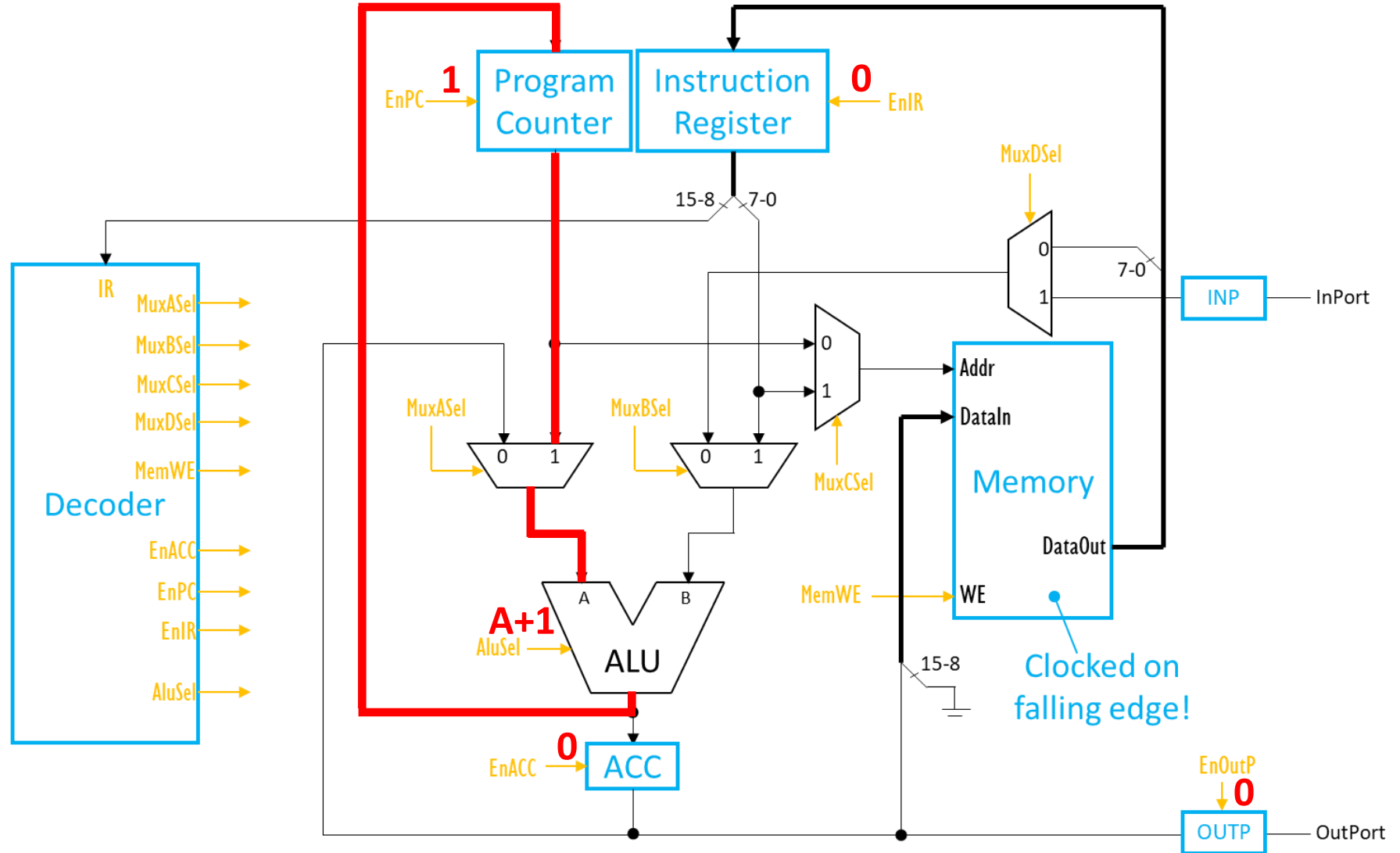
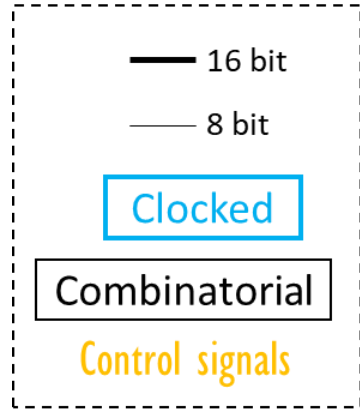
Simple CPU: decode (bv. "LOAD ACC 0x23")



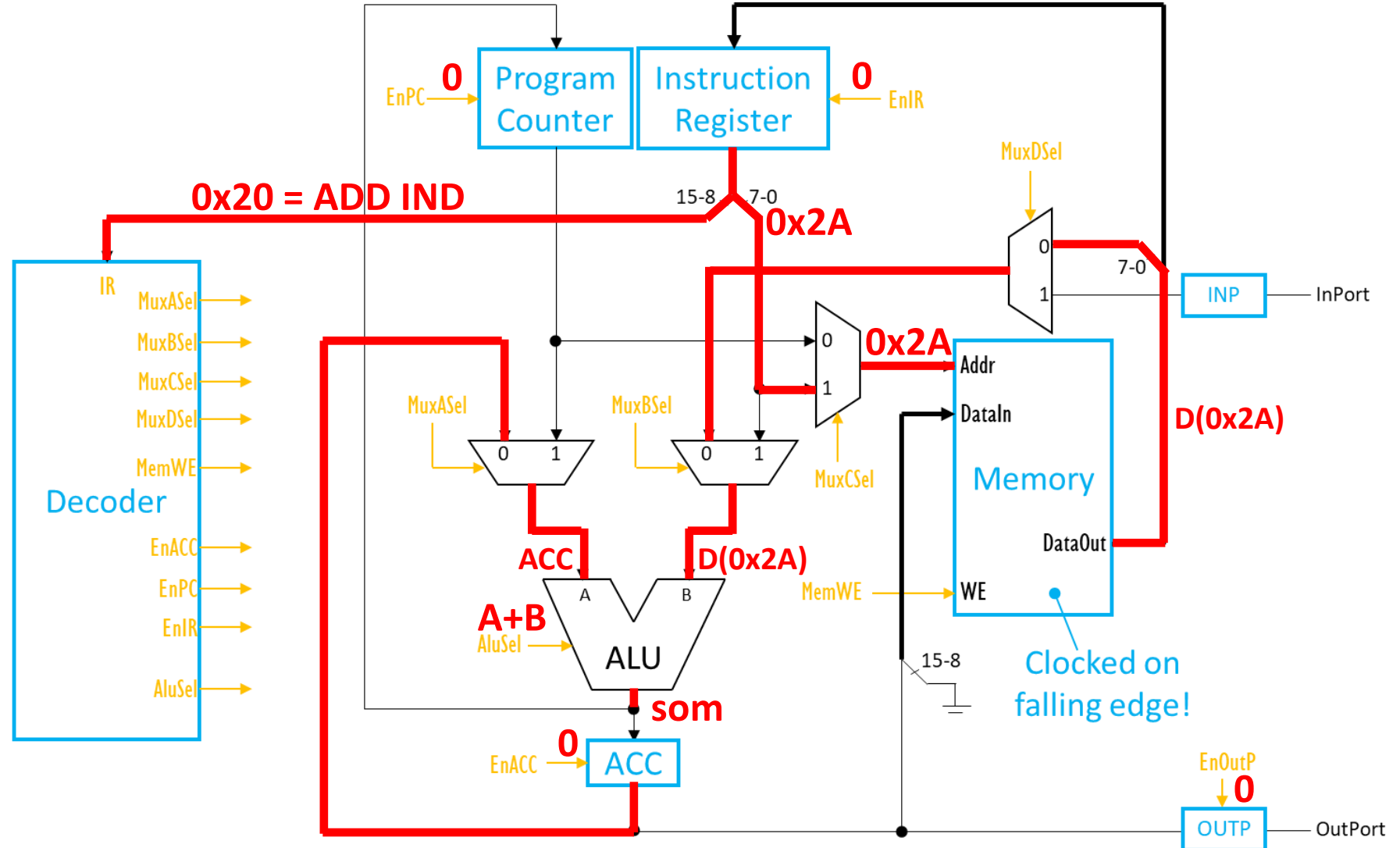
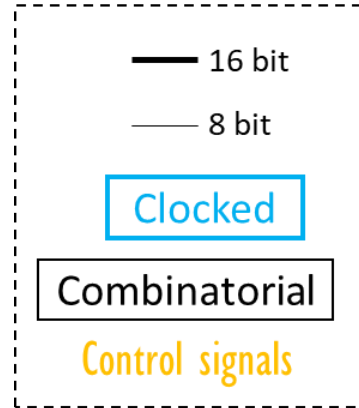
Simple CPU: execute (bv. "LOAD ACC 0x23")



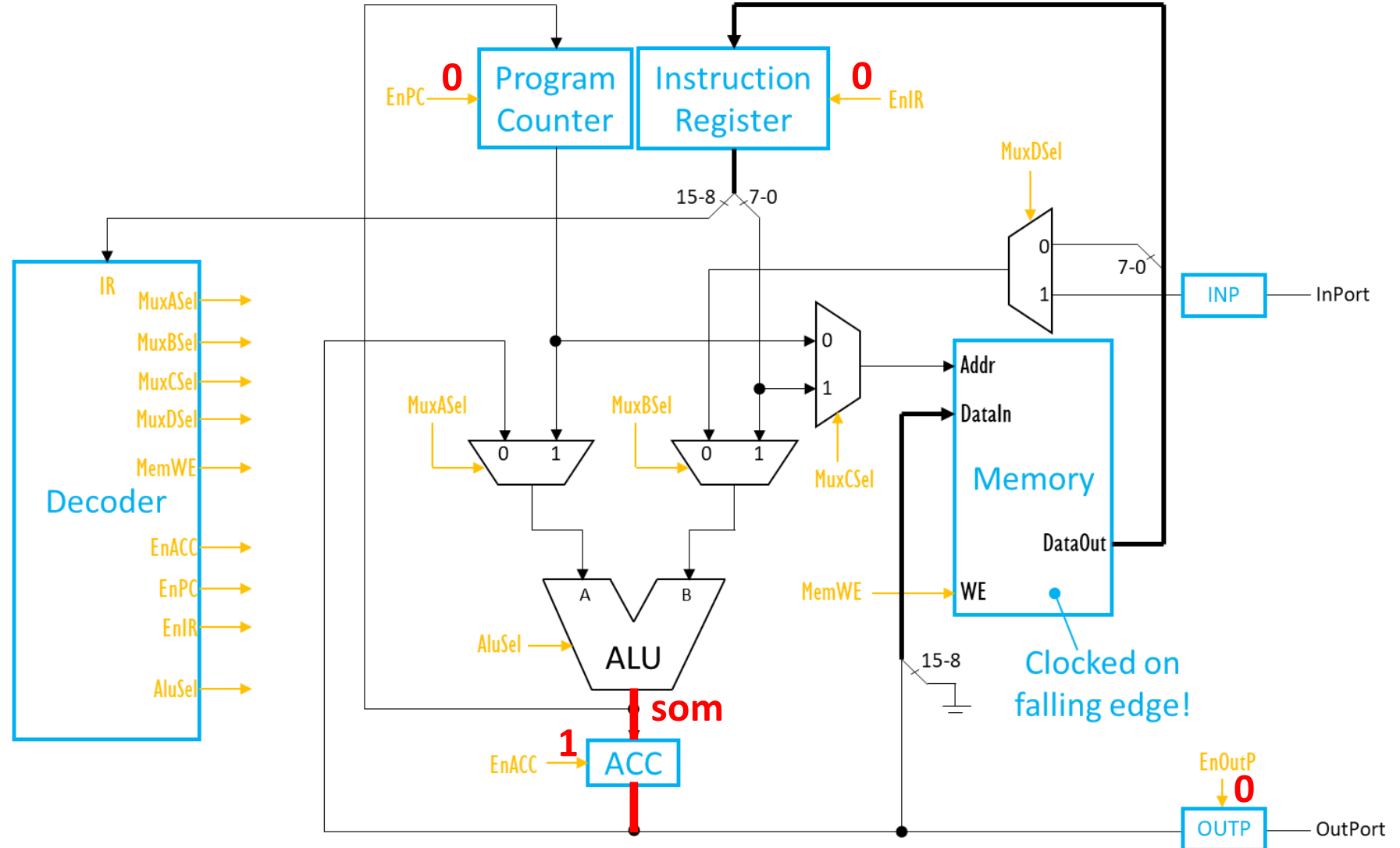
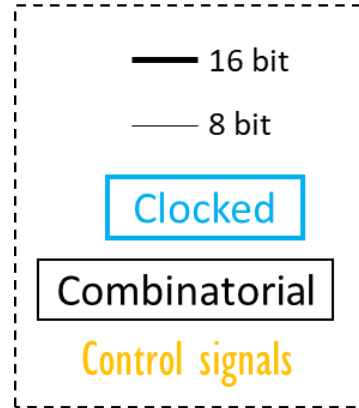
Simple CPU: increment



Simple CPU: decode (bv. "ADD IND 0x2A")



Simple CPU: execute (bv. "ADD IND 0x2A")



Module CPU hardware

■ Opdrachten:

1) werking standaard CPU: gegeven ASM/machinetaal

2) werking timer blok:

- bekijk VHDL
- teken blokschema: enkel in-/uitgangen + memory map

3) eigen instructie toevoegen aan CPU: BitOR

■ enkele assembler oefeningen, gericht op de hardware

4) timer blok

5) externe input / output

6) combinatie: gelijkaardig aan test laatste sessie!

Module CPU hardware: opdrachten

- **Niet inleveren:**
 - zelfdiscipline!
 - stel vragen!
 - laat eventueel controleren bij twijfel
- **Assembler vertalen naar machinetaal: routineklus**
 - gebruik HTML/javascript pagina
 - download van Blackboard (ASM2VHDL.zip)
 - schrijf programma adhv beschikbare instructies
 - genereer VHDL automatisch
 - plak VHDL in SimpleASM.vhd
 - mag gebruikt worden op test

Module CPU hardware: Tips

- Let op in VHDL: 16#12# betekent waarde 18, niet 12 (hex)!
- Instructies in SimpleASM.vhd = package => niet zichtbaar in hierarchie in Vivado => onder tab “Libraries”
- Bekijk ALLE demo's over Vivado op Mediasite ([channel Xilinx Vivado](#))!
- Lees de opdrachten aandachtig
- Test laatste zitting: tijdgebrek!!
 - => oefen veel + herhaal voor laatste sessie!
 - bereid je voor:
 - download gegeven SimpleCPU (Blackboard)
 - download ASM2VHDL webpagina
 - dien de juiste bestanden in!!! Zie demo's Vivado Mediasite!